

Class 16 Data Packets & 3 Way Handshake For SOC L1

IP/Data Packets

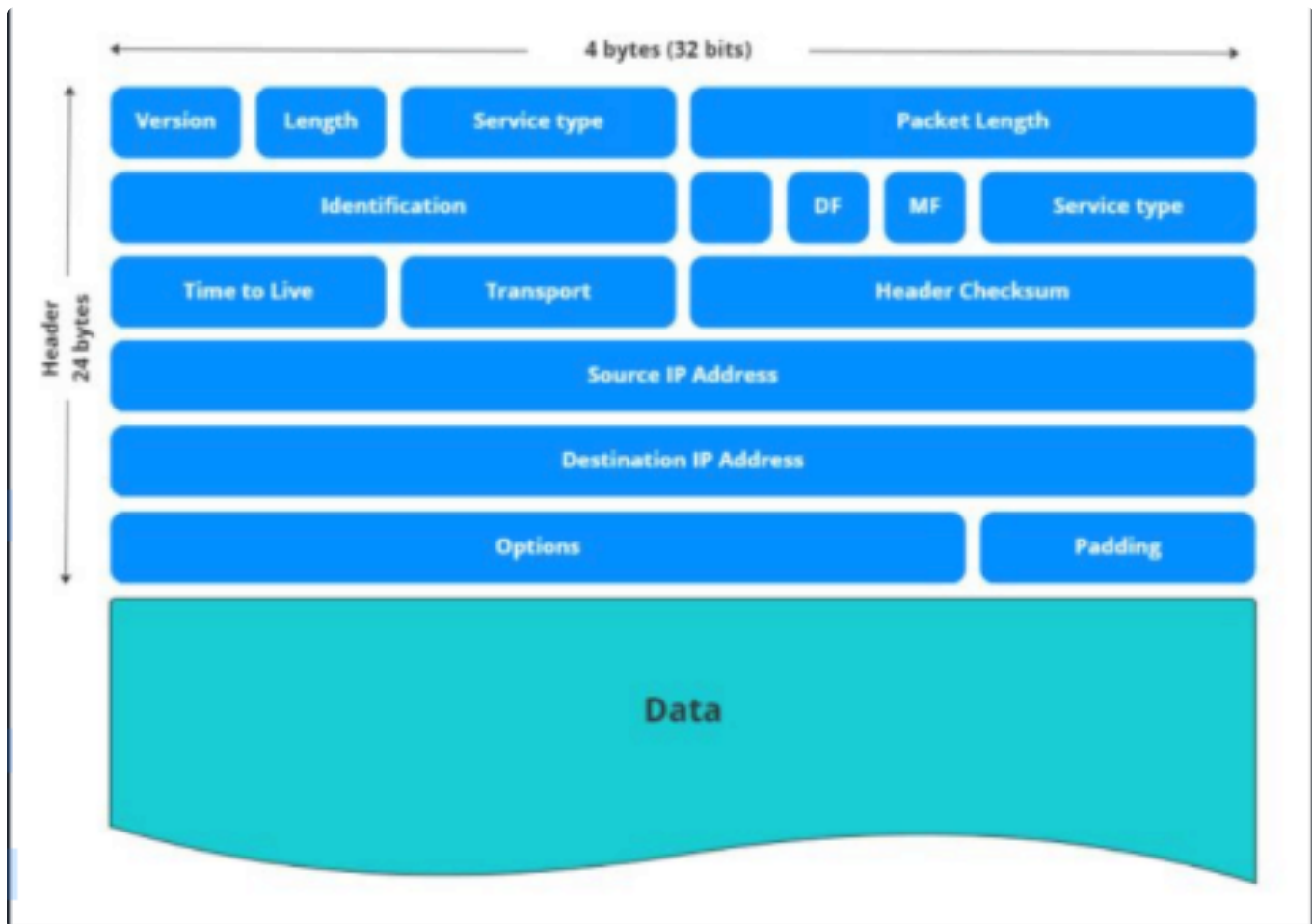
An **IP packet** is the **smallest unit of data** that is sent across a network using the Internet Protocol (IP).

Think of it like a **courier package**:

- **Payload (Data):** The actual information being sent (e.g., part of a web page, email, or file).
- **IP Header:** Contains important details such as:
 - **Source IP Address** – who sent the packet.
 - **Destination IP Address** – where the packet should go.
 - Other control information (such as TTL, protocol, and packet length) that helps routers deliver it correctly.

In one sentence:

An IP packet is the smallest unit of data transmitted over an IP network, containing both the actual data and a header with source, destination, and routing information needed to deliver the data to the correct destination.



A **packet** is the smallest unit of data transmitted over a network. Large data (like a file or webpage) is **broken into smaller packets**, sent across the network, and **reassembled** at the destination.

Structure of a Packet

Every packet has two parts:

- **Header** – Contains control information such as:
 - Source IP address
 - Destination IP address
 - Protocol
 - Time To Live (TTL)
 - Packet number/checksum
- **Payload** – The actual data being transmitted (text, image, video, etc.).

Packet Encapsulation

As data travels through the networking layers, **each layer adds its own header**.

```
Application Data
    ↓
Transport Layer → [TCP Header]
    ↓
Network Layer   → [IP Header]
    ↓
Data Link Layer → [Ethernet Header]

Final Packet:
[Ethernet Header] [IP Header] [TCP Header] [Data]
```

This process of adding headers at each layer is called **encapsulation**.

One-line Answer

An IP packet is the smallest unit of data sent over a network. It consists of a **header** (containing source, destination, TTL, protocol, etc.) and a **payload** (the actual data). As data moves through the network stack, each layer adds its own header, a process called **encapsulation**.

Structure of an IP Packet (IPv4 Header)

The **IP header** contains important information that helps routers deliver the packet to the correct destination.

- **Version** – Specifies the IP version being used (**IPv4** or **IPv6**).
- **Total Length** – Indicates the total size of the packet (header + data/payload).
- **Protocol** – Identifies which transport layer protocol carries the data, such as **TCP (6)**, **UDP (17)**, or **ICMP (1)**.
- **Time To Live (TTL)** – Limits how many routers (hops) the packet can pass through. Each router decreases the TTL by 1. When it reaches **0**, the packet is discarded to prevent routing loops.
- **Source IP Address** – The IP address of the sender.
- **Destination IP Address** – The IP address of the intended receiver. Routers use this to forward the packet.

Easy Way to Remember

Field	Purpose
Version	IPv4 or IPv6
Total Length	Size of the entire packet
Protocol	TCP, UDP, ICMP, etc.
TTL	Prevents infinite looping
Source IP	Sender's address
Destination IP	Receiver's address

IPv4 Header Fields - Layer 3 Network Layer

Field	Meaning	SOC Use
Version	IPv4 or IPv6	Protocol Identification
Source IP	Sender	Identity Attacker
Destination IP	Receiver	Target System
TTL (Time to Leave)	Packet Lifetime	Detect anomalies
Protocol	TCP/UPD/ICMP	traffic type
Total Length	Packet Size	Detect exfiltration
Header Checksum	Integrity	Corrupt detection

TCP Header : Layer 4

Field	Meaning	SOC Use
Source Port	Sender Port	Identity origin
Destination Port	Target service	Detect Attack
Sequence number	Order Tracking	Session analysis
ACK Number	Confirmation	Flow validation
Flags	Connection State	Attack Detection
Windows Size	Flow Control	Traffic behavior

Example of IP Packet

```
Source IP:      192.168.1.10
Destination IP: 45.77.123.66
Protocol:       TCP
TTL:           64
Length:        1500 bytes
```

What Each Field Means

- **Source IP:** 192.168.1.10 - The device that sent the packet.
- **Destination IP:** 45.77.123.66 - The device or server receiving the packet.
- **Protocol:** TCP - The transport protocol used for reliable communication.
- **TTL:** 64 - The packet can pass through up to 64 routers (hops). Each router decreases it by 1.
- **Length:** 1500 bytes - The total size of the packet (header + payload).

SOC Analyst Perspective

When investigating network traffic, a SOC analyst checks these fields to determine whether the traffic is legitimate or suspicious.

Field	Normal	Suspicious
Source IP	Internal/trusted IP	Unknown external IP or blacklisted IP
Destination IP	Company server or known trusted service	Malicious IP, unknown server, C2 server
Protocol	Expected protocol (TCP, UDP, ICMP)	Unusual protocol for the environment
TTL	Typical values (64, 128, 255)	Unusual values may indicate spoofing or scanning (requires context)
Length	Normal packet size	Extremely small or unusually large packets may require investigation

Example Analysis

Packet:

```
Source IP:      192.168.1.10
Destination IP: 45.77.123.66
Protocol:       TCP
TTL:           64
Length:        1500 bytes
```

SOC Investigation:

1. **Source IP:** 192.168.1.10 is a **private/internal IP** → likely an internal device.
2. **Destination IP:** Check if 45.77.123.66 is a **trusted server** or has a **malicious reputation** using threat intelligence.
3. **Protocol:** TCP is common for web, email, SSH, etc. Verify the destination port and application.
4. **TTL:** 64 is a normal initial TTL used by many Linux systems.
5. **Length:** 1500 bytes is the typical Ethernet Maximum Transmission Unit (MTU), so it's normal.

TTL - Time to Live

TTL (Time To Live) is a field in the **IP packet header** that limits how many **routers (hops)** a packet can pass through before it is discarded.

Its main purpose is to **prevent packets from looping forever** in the network due to routing errors.

How TTL Works

1. The sender assigns an initial TTL value (e.g., **64, 128, or 255**).
2. Every time the packet passes through a **router**, the router **decreases the TTL by 1**.
3. If the TTL reaches **0**, the router **drops the packet** and usually sends an **ICMP Time Exceeded** message back to the sender.

Example

```
Initial TTL = 64
```

```
Router 1 → TTL = 63
```

```
Router 2 → TTL = 62
```

```
Router 3 → TTL = 61
...
TTL = 0 → Packet is discarded
```

Why is TTL Important?

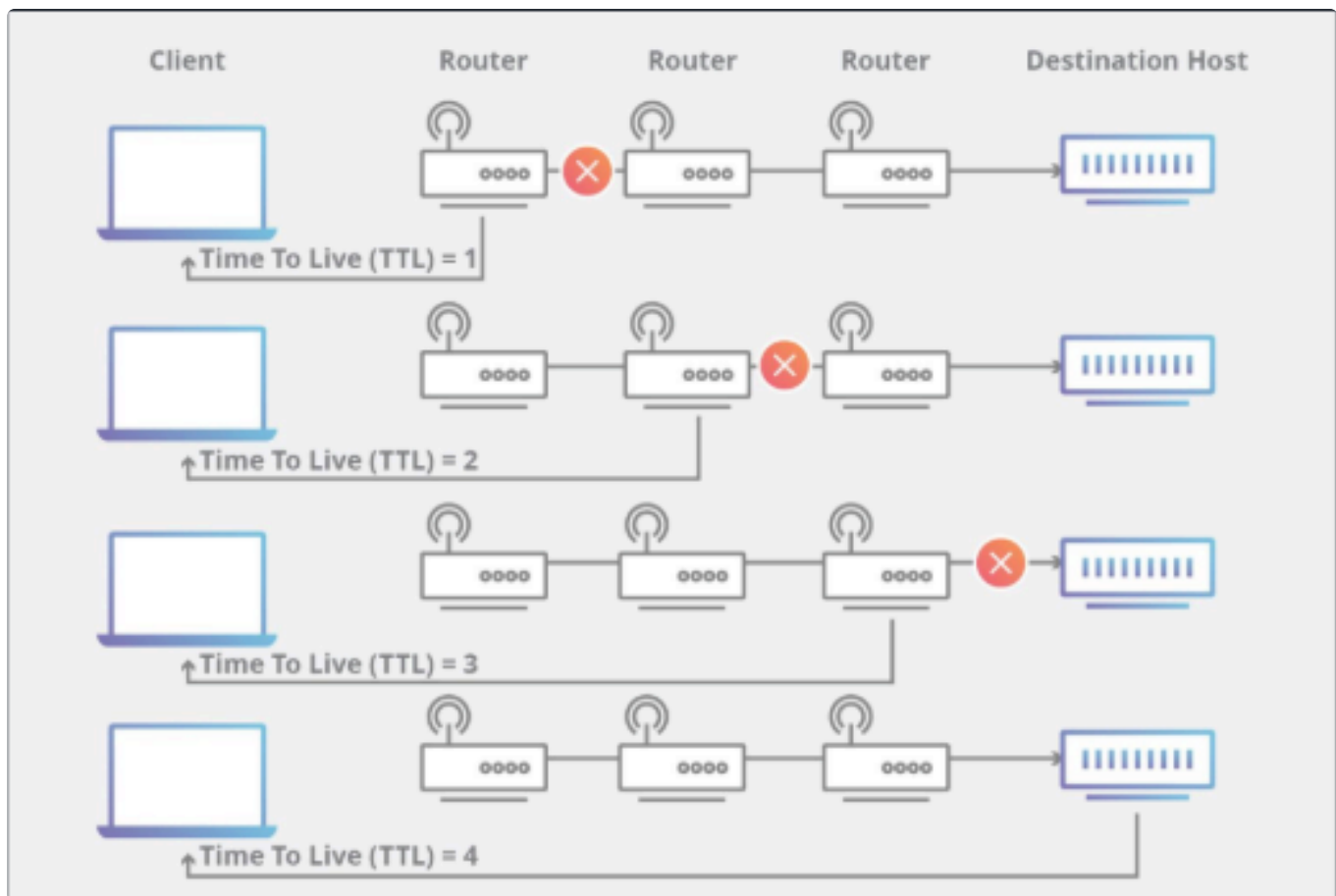
- Prevents **infinite routing loops**.
- Reduces **network congestion**.
- Helps diagnose network paths using tools like **tracert**.

SOC Analyst Perspective

As a SOC analyst, TTL can provide useful clues:

- **Normal TTL values:** 64, 128, 255 (depends on the operating system).
- A **low TTL** may indicate the packet has traveled through many routers.
- **Unexpected TTL values** can sometimes suggest **IP spoofing**, **network scanning**, or unusual routing, but TTL alone is **not evidence of an attack**. It should always be correlated with other logs and indicators.

TTL (Time To Live) is an 8-bit field in the IP header that limits how many routers, or hops, a packet can travel through. Each router decreases the TTL by 1. When the TTL reaches 0, the packet is discarded, preventing infinite routing loops and reducing network congestion. In SOC analysis, TTL values can help identify abnormal network behavior when correlated with other indicators.



How TTL Works

TTL (Time To Live) is a value in the **IP packet header** that limits how long a packet can travel across a network.

Step-by-Step

1. The sender creates a packet with an initial TTL value (e.g., **64**).
2. The packet reaches **Router 1**, which decreases the TTL by **1** (64 → 63).
3. **Each router** the packet passes through continues to decrease the TTL by **1**.
4. If the packet reaches its destination **before TTL becomes 0**, it is delivered successfully.
5. If the **TTL reaches 0**, the router:
 - **Discards (drops)** the packet.
 - Sends an **ICMP Time Exceeded** message back to the sender.

Example

```
Sender (TTL = 64)
  |
  ▼
Router 1 → TTL = 63
  |
  ▼
Router 2 → TTL = 62
  |
  ▼
Router 3 → TTL = 61
  |
  ▼
Destination ✓
```

If the packet keeps looping:

```
Sender (TTL = 3)
  |
  ▼
Router 1 → TTL = 2
  |
  ▼
Router 2 → TTL = 1
  |
  ▼
Router 3 → TTL = 0
  |
  ▼
Packet Dropped ✖
ICMP "Time Exceeded" sent to sender
```

Why TTL is Important

- Prevents **infinite routing loops**.
- Reduces **network congestion**.
- Helps tools like **tracert** discover the path packets take by intentionally sending packets with increasing TTL values.

SOC Analyst Perspective

When analyzing network traffic:

- A packet with a **normal TTL** (64, 128, or 255) is generally expected.

- A **TTL of 0** means the packet expired before reaching its destination.
- Frequent **ICMP Time Exceeded** messages or many expired packets may indicate:
 - Routing loops
 - Network misconfiguration
 - Connectivity issues
 - Excessive hop count

TTL, or Time To Live, is an 8-bit field in the IP header that limits the number of routers (hops) a packet can traverse. Each router decreases the TTL by one. If the TTL reaches zero before the packet reaches its destination, the router drops the packet and sends an ICMP "Time Exceeded" message to the sender. This prevents packets from circulating indefinitely and helps avoid network congestion.

TTL in ping and traceroute

Both **ping** and **traceroute** use the **TTL field**, but they use it in different ways.

1. Ping

- **Ping** checks whether a destination host is reachable.
- It sends an **ICMP Echo Request** packet.
- If the destination is reachable, it replies with an **ICMP Echo Reply**.
- The **TTL value in the reply** can give an idea of how many routers (hops) the packet has crossed.

Example:

```
ping 8.8.8.8
```

```
Reply from 8.8.8.8:  
TTL = 117
```

2. Traceroute

Traceroute is used to discover the **path (routers/hops)** a packet takes to reach the destination.

How it works

Traceroute sends packets with **increasing TTL values**.

- **Packet 1:** TTL = 1
 - Router 1 decreases TTL to 0.
 - Router 1 drops the packet.
 - Router 1 sends an **ICMP Time Exceeded** message.
 - Traceroute records Router 1.
- **Packet 2:** TTL = 2
 - Router 1 → TTL = 1
 - Router 2 → TTL = 0
 - Router 2 drops the packet and sends ICMP.
 - Traceroute records Router 2.
- **Packet 3:** TTL = 3
 - Router 3 responds.
 - And so on until the destination is reached.

Example

```
TTL = 1 → Router 1 → ICMP Time Exceeded  
TTL = 2 → Router 2 → ICMP Time Exceeded  
TTL = 3 → Router 3 → ICMP Time Exceeded  
TTL = 4 → Destination reached ✓
```

Each ICMP reply tells traceroute:

- Which **router** responded.
- **How long** it took for the packet to travel to that router and back (Round Trip Time - RTT).

Why SOC Analysts Use Traceroute

A SOC analyst may use traceroute to:

- Identify the **network path** to a server.

- Troubleshoot **routing issues**.
- Detect where packets are being **dropped**.
- Investigate **high latency** or connectivity problems.

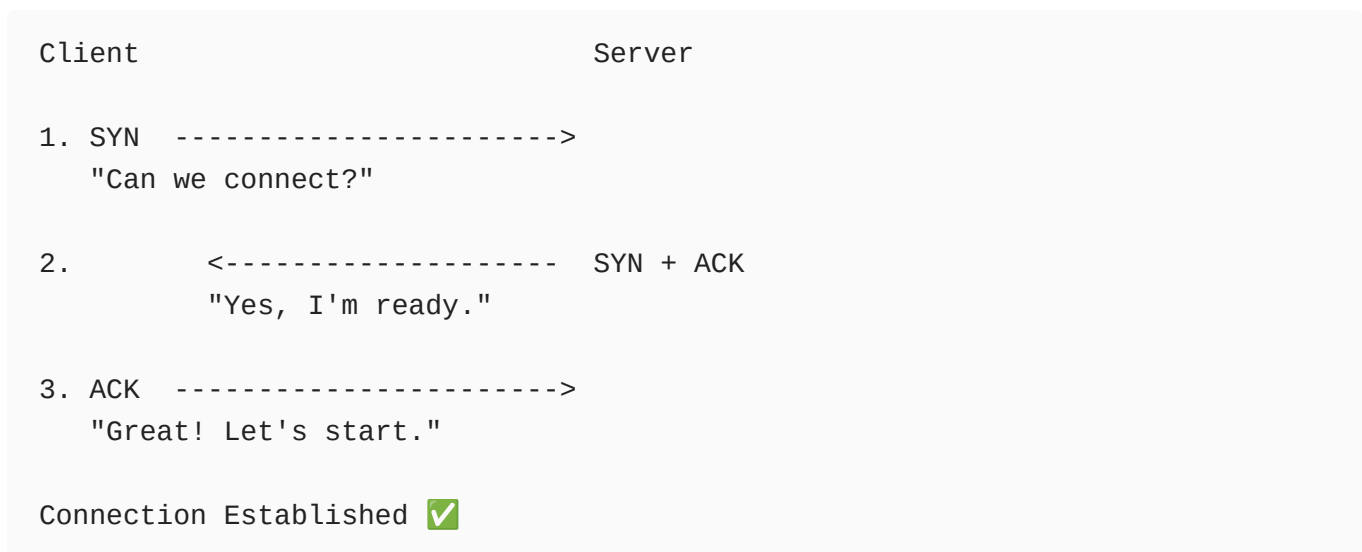
Ping uses ICMP to check whether a host is reachable and measures the round-trip time. **Traceroute** uses the TTL field by sending packets with increasing TTL values (1, 2, 3, ...). Each router decreases the TTL by one. When the TTL reaches zero, the router drops the packet and sends an **ICMP Time Exceeded** message. By repeating this process with higher TTL values, traceroute identifies each router along the path and measures the time taken to reach every hop.

TCP 3-Way Handshake

The **TCP 3-Way Handshake** is the process used to **establish a reliable connection** between a **client** and a **server** before any data is exchanged.

Its purpose is to ensure that **both devices are ready to communicate** and can send and receive data reliably.

The 3 Steps



Step 1: SYN (Synchronize)

- The **client** sends a **SYN** packet to the server.
- It requests to start a TCP connection.

Step 2: SYN-ACK (Synchronize + Acknowledge)

- The **server** receives the SYN packet.
- It replies with **SYN-ACK**, meaning:
 - **SYN**: "I also want to establish a connection."
 - **ACK**: "I received your SYN."

Step 3: ACK (Acknowledge)

- The **client** sends an **ACK** packet.
- This confirms that it received the server's SYN-ACK.

✅ The TCP connection is now established, and data transfer can begin.

Real-Life Analogy

Imagine making a phone call:

- **Client**: "Hello, can you hear me?" (**SYN**)
- **Server**: "Yes, I can hear you. Can you hear me?" (**SYN-ACK**)
- **Client**: "Yes, I can hear you too." (**ACK**)

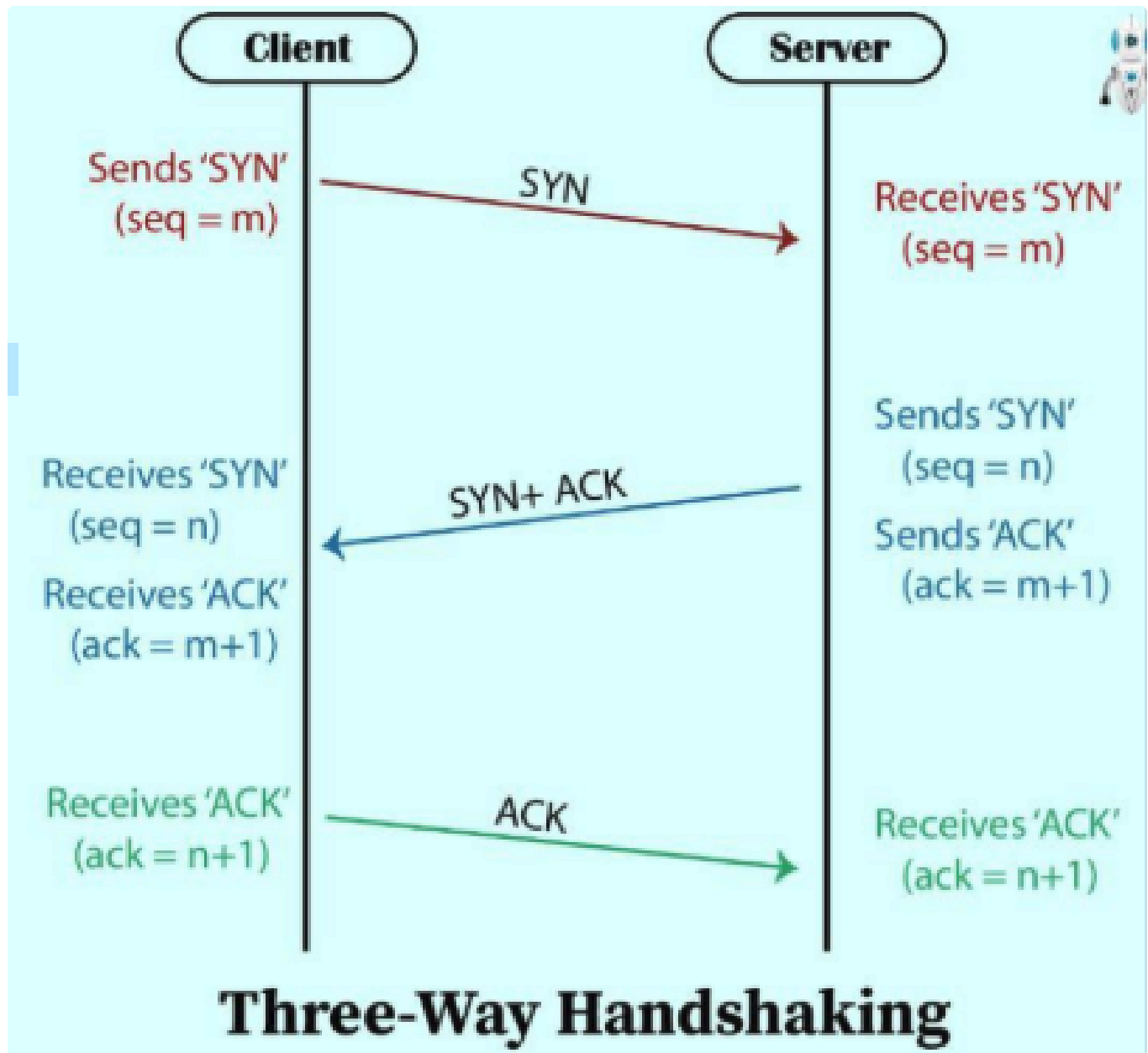
Now both people can start talking.

SOC Analyst Perspective

A SOC analyst monitors the TCP handshake to detect:

- **Successful connections** (normal traffic).
- **Incomplete handshakes**, which may indicate:
 - Network issues.
 - Firewall blocking traffic.
 - **SYN Flood (DoS/DDoS) attacks**, where attackers send many SYN packets but never complete the handshake, exhausting server resources.

The **TCP 3-Way Handshake** is the process used to establish a reliable TCP connection between a client and a server. First, the client sends a **SYN** packet to request a connection. The server responds with a **SYN-ACK** packet to acknowledge the request and indicate it is ready. Finally, the client sends an **ACK** packet to confirm receipt. After these three steps, the TCP connection is established, and reliable data transfer begins.



3 Way Handshake

Flag	Meaning	SOC Use
SYN	START	Connection attempts
ACK	Confirm	Normal Traffic
FIN	Close	Session End
RST	Reset	Abnormal Termination
PSH	Push Data	Immediate delivery
URG	Urgent	Rare

Suspicious Pattern For SOC

Pattern	Meaning
SYN only	SYN Flood
SYN + FIN	Malicious
RST Flood	Disruption
No ACK	Half-Open Attack

Real SOC Scenario

Alert:

```
Source IP:      185.221.x.x
Packets:       100,000 SYN
ACK Received:   0
Destination:   Port 80 (HTTP)
```

SOC Analysis

1. Validate

- Is the alert genuine?
- Check firewall, IDS/IPS, or SIEM logs.
- Confirm that a large number of **SYN packets** are being sent.

2. Investigate

- Source IP: 185.221.x.x (external IP).
- Destination: **Port 80 (HTTP)**.
- **100,000 SYN packets** sent in a short time.
- **No ACK packets** received to complete the TCP handshake.

3. Correlate

- Check if multiple external IPs are targeting the same server.
- Review firewall, web server, and network logs.
- Compare with historical traffic to determine if this is abnormal.

4. Enrich

- Check the source IP using threat intelligence.
- Look for previous malicious activity or reputation.

- Determine if other organizations have reported the IP.

5. Decide

- This is **not normal web traffic**.
- The attacker sends many **SYN** packets but never completes the TCP 3-way handshake.
- The server keeps half-open connections waiting for the final ACK, consuming resources.

6. Escalate

- Escalate to the Incident Response or Network team.
- Mitigation may include:
 - Blocking the malicious IP(s).
 - Enabling **SYN cookies**.
 - Applying rate limiting.
 - Activating DDoS protection (e.g., CDN/WAF).

Why is it a SYN Flood?

A normal TCP connection looks like this:

```
Client                Server

SYN  ----->
      <----- SYN-ACK
ACK  ----->

Connection Established ✓
```

A SYN Flood attack looks like this:

```
Attacker              Server

SYN  ----->
SYN  ----->
SYN  ----->
SYN  ----->
100,000 times...

(No ACK sent)
```

Server waits for ACK ❌
Half-open connections increase
Resources get exhausted

The logs show an external IP sending **100,000 SYN packets** to **port 80** without completing the TCP handshake, as no ACK packets are received. This creates a large number of **half-open TCP connections**, consuming server resources and preventing legitimate users from connecting. Based on this behavior, the activity is identified as a **SYN Flood DDoS attack**.